Programación I

Estructuras de Repetición: El Ciclo Exacto (FOR)

Aprenderemos a gestionar nuestros algoritmos utilizando estructuras que permiten ejecutar un conjunto de instrucciones una cantidad determinada de veces. Esto evita la repetición innecesaria de código y permite procesar grandes volúmenes de datos de manera eficiente.

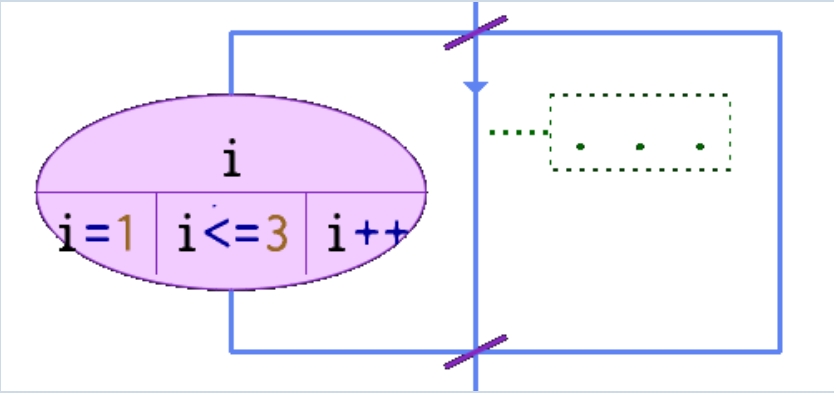
Introducción

Hasta ahora, si queríamos procesar 3 números, repetíamos las instrucciones 3 veces. Pero ¿qué pasaría si tuviéramos que procesar 100 o 1.000 datos?
Las **estructuras repetitivas (bucles)** resuelven este problema permitiendo que un bloque de código se ejecute cíclicamente. Se dividen en:

- **Ciclos Exactos:** Sabemos de antemano cuántas veces se va a repetir (ej. "procesar 10 notas").
- **Ciclos Inexactos:** No sabemos de antemano cuántas veces se va a repetir. La repetición depende de una condición que puede variar (ej. "leer números hasta que el usuario ingrese un cero").

El Ciclo Exacto (FOR)

El ciclo exacto nos permite ejecutar un conjunto de instrucciones una determinada cantidad de veces.



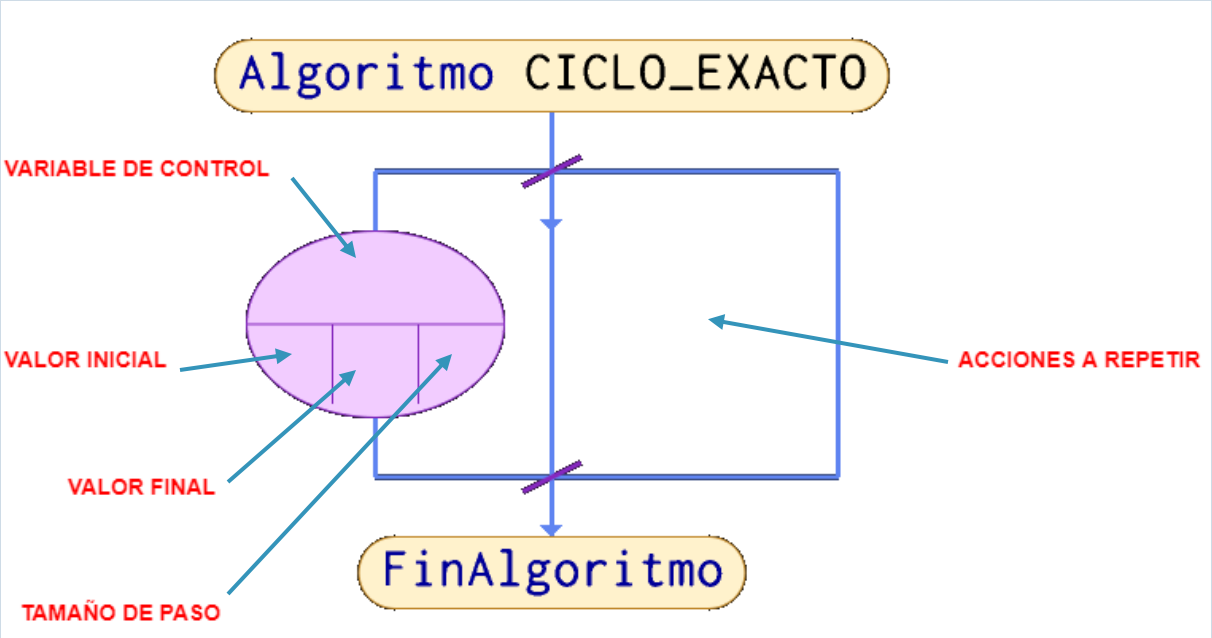
Diagrama

```
int i;
for (i=1; i<=3; i++){
/* Instrucciones a  repetir 3 veces.*/
}
```

Código C++

El Ciclo Exacto (FOR)

El ciclo exacto nos permite ejecutar un conjunto de instrucciones una determinada cantidad de veces.



El Ciclo Exacto (FOR)

Es una estructura que controla la repetición mediante una **variable de control** (usualmente llamada i).

Para que funcione, requiere tres componentes esenciales:

- Inicialización o Valor Inicial: ¿Desde dónde empieza a contar? (ej. i = 1).
- Comparación o Valor Final (Tope): ¿Hasta dónde llega? (ej. i <= 3).
- incremento/decremento o Tamaño de Paso: ¿Cómo avanza en cada vuelta? (ej. de 1 en 1).

for (i=1; i<=3; i++)

i = 1

La **inicialización** determina con qué valor comienza la variable que controlará el ciclo exacto. Esta instrucción se ejecuta una sola vez al comenzar el ciclo.

i <= 3

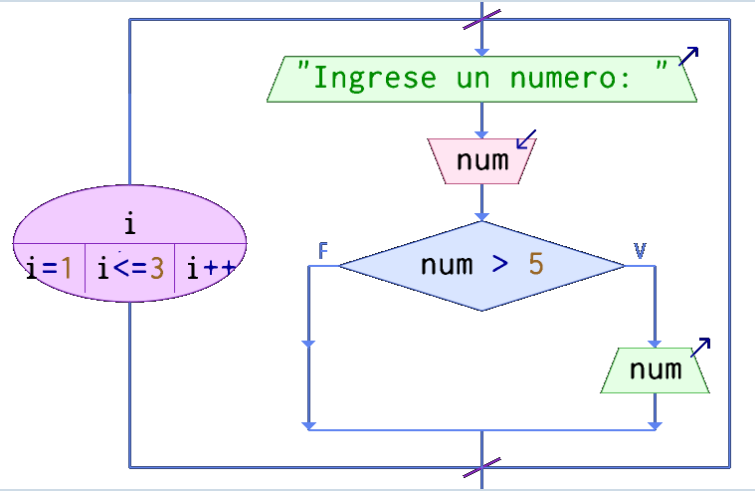
La **comparación** determina si el ciclo exacto debe seguir iterando o finalizar. Es una proposición lógica que debe ser *verdadera* para que el ciclo continúe iterando. Se evalúa una vez por cada iteración.

i++

El **incremento/decremento** es una instrucción que indica cuanto aumentará o disminuirá la variable que controla el ciclo. Se ejecuta una vez por cada vuelta del ciclo.

Ejemplo Práctico

Problema: Hacer un programa que permita ingresar 3 números enteros y que luego se informe por pantalla aquellos que sean mayores a 5.



Diagrama

```
int num, i;
for (i=1; i<=3; i++){
   cout << "Ingrese um numero: ";
   cin >> num;
   if (num > 5) {
      cout << num << endl;
   }
}
```

Código C++

Análisis del Comportamiento (Prueba de Escritorio)

¿Qué pasa internamente en cada "vuelta" o iteración?

Vuelta del ciclo	i (variable de control)	Resultado de la decisión (i<=3) (donde 3 es el valor final)	Tamaño de paso (incremento i en 1)
1	1	VERDADERO	i=i+1 (i vale 2)
2	2	VERDADERO	i=i+1 (i vale 3)
3	3	VERDADERO	i=i+1 (i vale 4)
CORTE	4	FALSO (FIN DEL CICLO)	

El ciclo termina cuando la variable de control es i = 4. El programa continúa con la instrucción siguiente al for.

Errores comunes

- ✓ **Punto y coma fatal**: Escribir for(...); { ... }. El ; después del paréntesis cierra el ciclo vacío y el bloque dentro del for se ejecuta una sola vez.

- ✓ **Sentido del paso**: Si el inicio es 1 y el fin es 10, pero el paso es -1, el ciclo nunca terminará (bucle infinito) o nunca empezará.

- ✓ **Modificar la variable de control**: Cambiar el valor de i dentro del cuerpo del for es una mala práctica que desvirtúa la lógica del ciclo exacto.